

K-Means ve Hiyerarşik Kümeleme Algoritmalarının Karşılaştırması ve Veri Ölçeklendirmenin Performans Üzerindeki Etkisi

Salih Özbek, Emre Karaaydın

25.10.2025

1. Özet

Bu çalışma, İris veri seti üzerinde iki temel gözetimsiz kümeleme algoritmasının (K-Means ve Hiyerarşik Kümeleme) performansını, iki farklı veri ön işleme senaryosu (Ham Veri ve Ölçekli Veri) altında karşılaştırmaktadır. Analiz, algoritmaların temel çalışma prensiplerindeki farkları ve veri yapısına verdikleri tepkileri ortaya koymayı amaçlamaktadır.

Temel Bulgular:

1. Algoritma Performansı: Elde edilen en iyi senaryo olan ham (ölçeksiz) veri üzerinde, K-Means ve Hiyerarşik Kümeleme algoritmaları neredeyse aynı ve yüksek bir başarı sergilemiştir. İki model de 150 örnek içerisinde toplamda 16'şar hata yapmıştır.
2. Ön İşlemenin Etkisi: İris veri seti üzerinde yapılan veri ölçeklendirmenin her iki modelin performansında da olumsuz etki verdiği gözlemlenmiştir. İrisin 4 özneliliğinin de zaten aynı cinsten (cm) olması sebebiyle verinin ham yapısının modeller için daha iyi olduğu tespit edilmiştir.
3. Algoritma Dayanıklılığı: Hatalı ön işleme senaryosunda iki model arasındaki en keskin fark açığa çıkmıştır. Ölçekli veri üzerinde; K-Means algoritmasının hata sayısı 25'e çıkarak performansta düşüş yaşanırken, Hiyerarşik Kümeleme algoritmasının performansı tamamen çökmüştür. Bu, K-Means algoritmasının bu senaryoda daha dayanıklı bir algoritma olduğunu göstermektedir.

2. Giriş

Gözetimsiz öğrenme, etiketlenmemiş veri setlerindeki doğal grupları ve yapıları keşfetme sürecidir. Bu çalışmada, bu alanın en bilinen iki algoritması olan K-Means ve Hiyerarşik Kümeleme algoritmaları, klasik İris veri seti üzerinde test edilmiştir. İris veri seti, 3 farklı lale türünden (Setosa, Versicolor, Virginica) 50'şer adet olmak üzere toplam 150 örnek içermektedir. Rapor, bu iki algoritmanın temel çalışma prensiplerini karşılaştıracak ve StandardScaler ile yapılan ön işlemenin bu iki farklı mantık üzerindeki etkisini analiz edecektir.

3. Algoritmaların Çalışma Prensipleri

Karşılaştırılan iki model kümeleri bulmak için farklı yollar kullanır.

Özellik	K-Means	Hiyerarşik Kümeleme
Temel Felsefe	Bölücü: Veriyi k adet kümeye böler.	Birleştirici: Her noktayı bir küme sayarak benzerleri birleştirir.
Çalışma Mantığı	1. k adet rastgele küme merkezi seçer. 2. Her veri noktasını kendisine en yakın merkezin kümesine atar. 3. Oluşan yeni kümelerin merkezlerini günceller 4. Merkezler sabitlenene kadar 2. ve 3. Adımları tekrarlar	1. Başlangıçta her veri noktası bir kümedir. 2. Birbirine en yakın iki kümeyi bulur ve birleştirir. 3. Tek bir küme kalana kadar 2. adımı tekrarlar. 4. Alınan k sayısına göre oluşan ağacı keser
Parametre	n_cluster (k) : küme sayısı random_state: rastgelelik seed'i n_init: modelin kaç defa çalıştırılacağını belirler	linkage: Kümelerin benzerliğinin nasıl ölçüleceğini belirleyen parametre. Bu çalışmada varsayılan olan ward metodu kullanılmıştır.

4. Deney Tasarımı

Analiz için 2 model ve 2 veri tipi ile tomlampda 4 farklı senaryo tasarlanmıştır.

Değerlendirme modellerin bulduğu kümelerin, gerçek tür etiketleriyle “pandas.crosstab” kullanılarak karşılaştırılmasıyla yapılmıştır. Amaç, bulunan her bir kümenin ne kadar saf (tek bir gerçek kümeye ait) olduğunu görmektir.

Veri Tipleri:

1. Ham (ölçeksiz) veri: x=iris.data
2. Ölçekli veri: x_scaled = scaler.fit_transform(x)

Modeller:

1. K-Means: KMeans(n_clusters=3, random_state=42 , n_init = 10)
2. Hiyerarşik Kümeleme: AgglomerativeClustering(n_clusters=3)

Bu bir sınıflandırma problemi değil, bir kümeleme problemi olduğu için veri seti bölünmemiştir. Amaç; modelin, verinin tamamının doğal yapısını en iyi şekilde keşfetmesini sağlamaktır.

5. Bulgular

5.1 K-Means Performans Analizi

K-Means, verinin nasıl sunulduğuna bağlı olarak farklı sonuçlar vermiştir.

Ham Veri: Model, `kmeans.fit_predict(x)` komutuyla çalıştırılmıştır. `pd.crosstab` ile çıktısı alınmıştır. Modelin Setosa'yı mükemmel bulduğunu (0 hata), ancak Versicolor (2 hata) ve Virginica (14 hata) türlerini karıştırdığı görülmüştür.

Ölçekli Veri: Aynı model `x_scaled` (ölçekli veri) ile çalıştırıldığında, hata sayısında artış gözlenmiştir.

```
Sütunlar: K-Means'in bulduğu kümeler (0, 1, 2)
Satırlar: Gerçek Türler (0: Setosa, 1: Versicolor, 2: Virginica)

K-Means (ölçeksiz) Karşılaştırma Tablosu:
col_0  0  1  2
row_0
0       0 50  0
1      48  0  2
2      14  0 36

K-Means (ölçekli) Karşılaştırma Tablosu:
col_0  0  1  2
row_0
0       0 50  0
1      39  0 11
2      14  0 36
```

5.2 Hiyerarşik Kümeleme Performans Analizi

Hiyerarşik Kümeleme algoritmasının veri ön işlemeye tepkisi daha keskin olmuştur.

Ham Veri: Model; `agg_cluster.fit_predict(x)` komutuyla çalıştırılmıştır. `pd.crosstab` çıktısı, Setosa'yı mükemmel bulduğunu (0 hata), Versicolor (1 hata) ve Virginica (15 hata) türlerini ise K-Means'e çok benzer bir şekilde karıştırdığını göstermiştir.

Ölçekli Veri: Aynı model `x_scaled` (ölçekli veri) üzerinde çalıştırıldığında modelin anlamlı kümeler bulmakta zorlandığı görülmüştür.

```
Sütunlar: Hiyerarşik Kümeleme'nin bulduğu kümeler (0, 1, 2)
Satırlar: Gerçek Türler (0: Setosa, 1: Versicolor, 2: Virginica)

Hiyerarşik Kümeleme (ölçeksiz) Karşılaştırma Tablosu:
col_0  0  1  2
row_0
0       0 50  0
1      49  0  1
2      15  0 35

Hiyerarşik Kümeleme (ölçekli) Karşılaştırma Tablosu:
col_0  0  1  2
row_0
0       0 49  1
1      23  0 27
2      48  0  2
```

6. Sonuç

Bu analiz iki algoritma ve iki ön işleme senaryosunu karşılaştırarak net sonuçlar ortaya koymuştur.

1. Ön İşlemenin Etkisi

En önemli bulgu ön işlemenin performans üzerindeki etkisidir. İrisin dört özniteliğinin de benzer birimde (cm) olması sebebiyle, ölçeklendirme işlemi verinin doğal ve ayırt edici yapısını bozmuş ve her iki modelin de performansını düşürmüştür. Kullanılan bu veri seti için en iyi yöntemin ham veriyi kullanmak olduğu sonucuna varılmıştır.

2. K-Means ve Hiyerarşik Kümeleme'nin Karşılaştırılması

En iyi senaryolarında iki model de aynı oranda başarı göstermiştir (16 hata). Hata dağılımlarının (14+2 , 15+1) çok benzer olması, her iki modelin de veri seti içerisindeki aynı 16 **zorlu** çiçeği ayırmakta zorlandığını göstermektedir. Bu senaryoda iki model de performans açısından eşdeğerdir.

İki model arasındaki asıl fark, yanlış veri ön işleme sonucunda ortaya çıkmıştır. K-Means'in performansı bu durumda yalnızca belli bir oranda bozulurken, Hiyerarşik Kümeleme bu senaryoda anlamlı bir sonuç üretememiştir. Bu, K-Means'in bu veri seti -İris- üzerinde daha dayanıklı ve stabil bir algoritma olduğunu göstermektedir.

Nihai Sonuç:

İris veri setindeki doğal grupları keşfetmek için en iyi yöntem, ham veriyi kullanmaktır. Bu senaryoda, K-Means ve Hiyerarşik Kümeleme performans açısından eşdeğerdir. Ancak K-Means algoritmasının farklı ön işleme senaryolarına karşı daha dayanıklı olduğu gözlenmiştir.